File Handling in Java

14 July 2026 16:52

File handling in Java allows us to create, read, write, update and delete files stored on the system.

What is File Handling in Java?
In Java, a file is a named in location on Secondary storage device used to store related information. File handling refers to the process of managing files programmatically., such as:

- Creating a file
- Retrieving file information.
- Writing data into a file.
- Reading data from a file.
- Deleting a file.

Java provides the **java.io** package to perform file handling operations.

Stream in Java
A stream represents a sequence of data. File handling is performed using screens which are classified into two types.

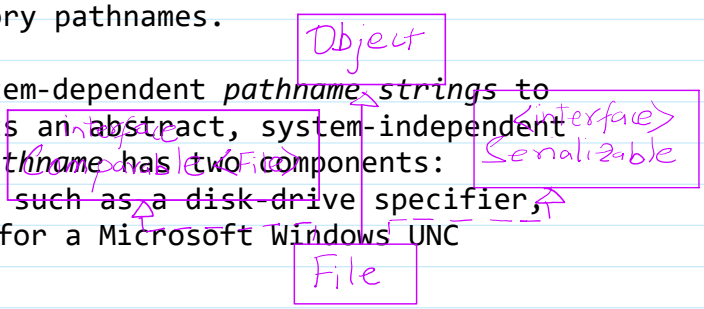
1. **Byte Stream:** White stream is used to handle byte oriented data such as images, audio files and binary sites. Classes like FileInputStream and FileOutputStream are used for byte stream operations.

2. **Character Stream:** Character stream is used to handle character oriented data such as text files. Classes like FileReader, FileWriter, BufferedReader, and BufferedWriter are used for character stream operation.

The File class in Java

public class File extends Object implements Serializable, Comparable<File>

An abstract representation of file and directory pathnames.



User interfaces and operating systems use system-dependent *pathname strings* to name files and directories. This class presents an abstract, system-independent view of hierarchical pathnames. An *abstract pathname* has two components:
1. An optional system-dependent *prefix* string, such as a disk-drive specifier, "/" for the UNIX root directory, or "\\\\" for a Microsoft Windows UNC pathname, and
2. A sequence of zero or more string *names*.

The first name in an abstract pathname may be a directory name or, in the case of Microsoft Windows UNC pathnames, a hostname. Each subsequent name in an abstract pathname denotes a directory; the last name may denote either a directory or a file. The *empty* abstract pathname has no prefix and an empty name sequence.
The conversion of a pathname string to or from an abstract pathname is inherently system-dependent. When an abstract pathname is converted into a pathname string,

each name is separated from the next by a single copy of the default *separator character*. The default name-separator character is defined by the system property file.separator, and is made available in the public static fields separator and separatorChar of this class. When a pathname string is converted into an abstract pathname, the names within it may be separated by the default name-separator character or by any other name-separator character that is supported by the underlying system.

A pathname, whether abstract or in string form, may be either *absolute* or *relative*. An absolute pathname is complete in that no other information is required in order to locate the file that it denotes. A relative pathname, in contrast, must be interpreted in terms of information taken from some other pathname. By default the classes in the java.io package always resolve relative pathnames against the current user directory. This directory is named by the system property user.dir, and is typically the directory in which the Java virtual machine was invoked.

The *parent* of an abstract pathname may be obtained by invoking the getParent() method of this class and consists of the pathname's prefix and each name in the pathname's name sequence except for the last. Each directory's absolute pathname is an ancestor of any File object with an absolute abstract pathname which begins with the directory's absolute pathname. For example, the directory denoted by the abstract pathname "/usr" is an ancestor of the directory denoted by the pathname "/usr/local/bin".

The prefix concept is used to handle root directories on UNIX platforms, and drive specifiers, root directories and UNC pathnames on Microsoft Windows platforms, as follows:

- For UNIX platforms, the prefix of an absolute pathname is always "/". Relative pathnames have no prefix. The abstract pathname denoting the root directory has the prefix "/" and an empty name sequence.
- For Microsoft Windows platforms, the prefix of a pathname that contains a drive specifier consists of the drive letter followed by ":" and possibly followed by "\\" if the pathname is absolute. The prefix of a UNC pathname is "\\\\"; the hostname and the share name are the first two names in the name sequence. A relative pathname that does not specify a drive has no prefix.

Instances of this class may or may not denote an actual file-system object such as a file or a directory. If it does denote such an object then that object resides in a *partition*. A partition is an operating system-specific portion of storage for a file system. A single storage device (e.g. a physical disk-drive, flash memory, CD-ROM) may contain multiple partitions. The object, if any, will reside on the partition named by some ancestor of the absolute form of this pathname.

A file system may implement restrictions to certain operations on the actual file-system object, such as reading, writing, and executing. These restrictions are collectively known as *access permissions*. The file system may have multiple sets of access permissions on a single object. For example, one set may apply to the object's *owner*, and another may apply to all other users. The access permissions on an object may cause some methods in this class to fail.

Instances of the File class are immutable; that is, once created, the abstract pathname represented by a File object will never change.

Interoperability with java.nio.file package

The java.nio.file package defines interfaces and classes for the Java virtual machine to access files, file attributes, and file systems. This API may be used to overcome many of the limitations of the java.io.File class. The toPath method

may be used to obtain a Path that uses the abstract path represented by a File object to locate a file. The resulting Path may be used with the Files class to provide more efficient and extensive access to additional file operations, file attributes, and I/O exceptions to help diagnose errors when an operation on a file fails.

Since:

1.0

From <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/io/File.html>

Java provides the File class in the java.io package to represent the path name of a file or directory.

The File class does not read or write file contents. Instead, it is used to:

- Create files.
- Delete files.
- Rename files
- Check whether a file exists.
- Retrieve file information.
- Create directories.
- List directory contents.

For reading and writing file contents, classes such as FileReader, FileWriter, FileInputStream and FileOutputStream are used.

Package: import java.io.File;

What is a File Object?

A **File** object represents the path of a file or directory.

D:\Java\Student.txt

The object stores the file path, but does not automatically create a file.

Creating a File Object

```
File file = new File("D:/Java/Student.txt");
```

Note: The constructor only creates a File object. The actual file is created using the createNewFile() method.

Constructor Summary

Constructors

Constructor	Description
`File(File parent, String child)`	Creates a new File instance from a parent abstract pathname and a child pathname string.
`File(String pathname)`	Creates a new File instance by converting the given pathname string into an abstract pathname.
`File(String parent, String child)`	Creates a new File instance from a parent pathname string and a child pathname string.
`File(URI uri)`	Creates a new File instance by converting the given file: URI into an abstract pathname.

`File(URI uri)`	Creates a new `File` instance by converting the given `file:` URI into an abstract pathname.

File dir = new File("D:\\Java");
File file = new File(dir, "student.txt");

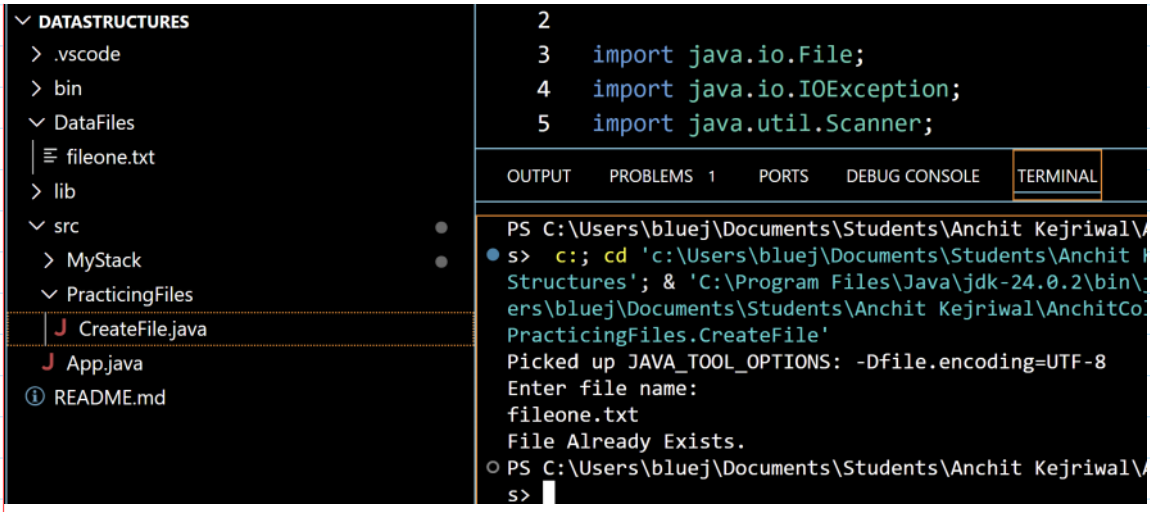
> File file = new File("D:\\Java", "student.txt");

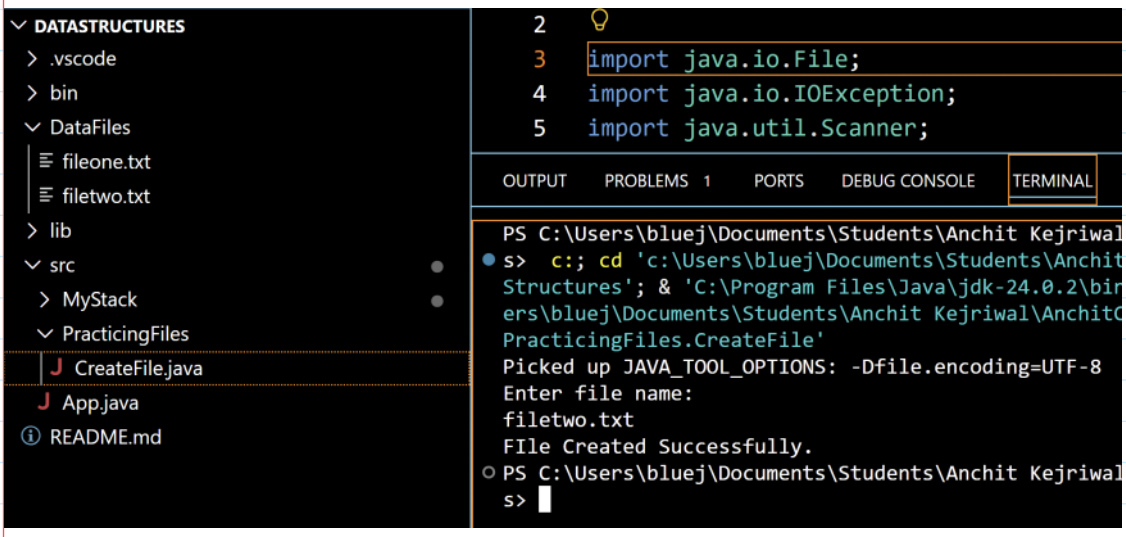
Creating a new File

The createNewFile() method creates a new empty file.

> File file = new File("student.txt");

```
package PracticingFiles;
import java.io.File;
import java.io.IOException;
import java.util.Scanner;
public class CreateFile {
    public static void main(String[] args) {
        String path = "DataFiles";
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter file name: ");
        String fileName = sc.nextLine();
        File file = new File(path, fileName);
        sc.close();
        try {
            if(file.createNewFile()){
                System.out.println("FIle Created Successfully.");
            }
            else{
                System.out.println("File Already Exists.");
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```





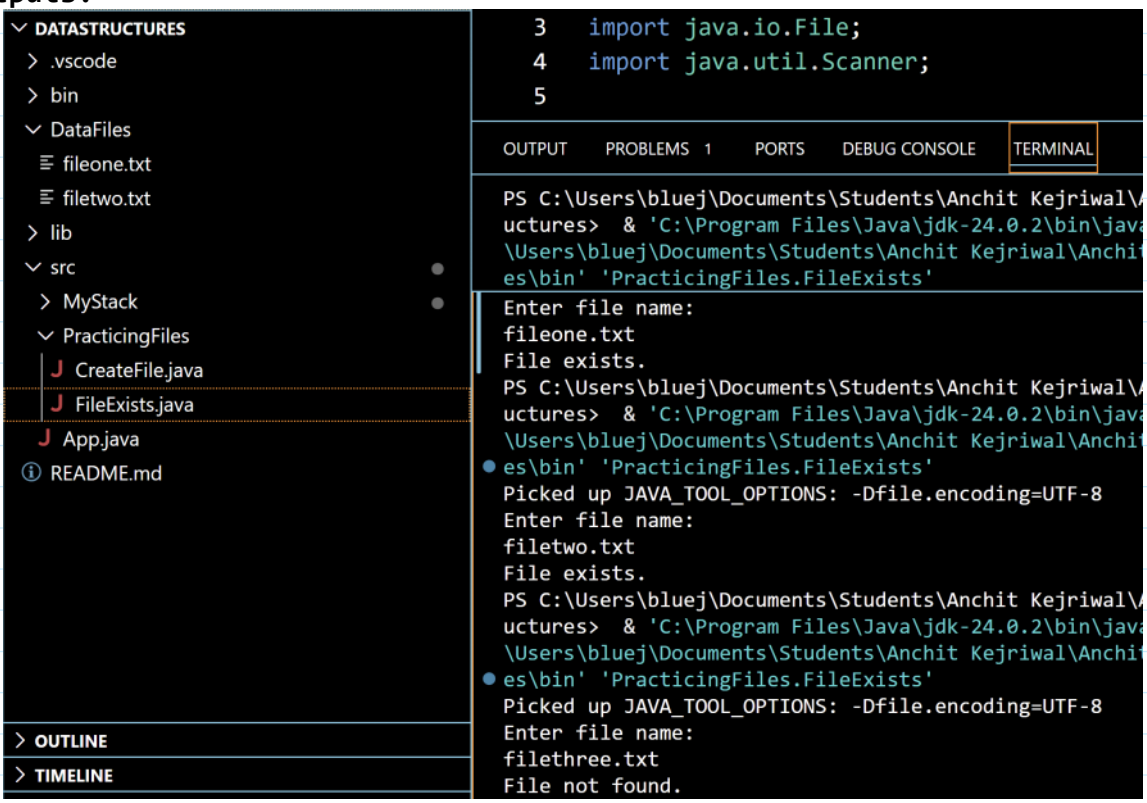
Checking Whether a File Exists

exists()

```java
package PracticingFiles;
import java.io.File;
import java.util.Scanner;
public class FileExists {
    public static void main(String[] args) {
        String path = "DataFiles";
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter file name: ");
        String fileName = sc.nextLine();
        File file = new File(path, fileName);
        sc.close();
        if(file.exists()){
            System.out.println("File exists.");
        }
        else{
            System.out.println("File not found.");
```

```
        }
    }
}
```

Outputs:



Getting File Name: getName()

```java
package PracticingFiles;
import java.io.File;
import java.util.Scanner;
public class FileExists {
    public static void main(String[] args) {
        String path = "DataFiles";
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter file name: ");
        String fileName = sc.nextLine();
        File file = new File(path, fileName);
        sc.close();
        if(file.exists()){
            System.out.println("File exists.");
            System.out.println("Name of existing file: "+ file.getName());
        }
        else{
            System.out.println("File not found.");
        }
    }
}
```

Output:

```
Enter file name:
filetwo.txt
File exists.
Name of existing file: filetwo.txt
```

Getting File Path: getPath()

```java
package PracticingFiles;
import java.io.File;
import java.io.IOException;
import java.util.Scanner;
public class FileExists {
    public static void main(String[] args) {
        String path = "DataFiles";
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter file name: ");
        String fileName = sc.nextLine();
        File file = new File(path, fileName);
        sc.close();
        if(file.exists()){
            System.out.println("File exists.");
            System.out.println("Name of existing file: "+ file.getName());
            System.out.println("Path: " + file.getPath());
            System.out.println("Absolute path: " + file.getAbsolutePath());
            try {
                System.out.println("Canonical path: "+ file.getCanonicalPath());
            } catch (IOException e) {
                e.printStackTrace();
            }

        }
        else{
            System.out.println("File not found.");
        }
    }
}
```

This method also returns Absolute path.

Output:

```
Enter file name:
fileone.txt
File exists.
Name of existing file: fileone.txt
Path: DataFiles\fileone.txt
Absolute path: C:\Users\bluej\Documents\Students\Anchit Kejriwal\AnchitCollege\Semester1\ForGit\Coding\JavaProj
ects\DataStructures\DataFiles\fileone.txt
Canonical path: C:\Users\bluej\Documents\Students\Anchit Kejriwal\AnchitCollege\Semester1\ForGit\Coding\JavaPro
jects\DataStructures\DataFiles\fileone.txt
```

Check Read permission: canRead()

Check Write permission: canWrite()

Check Execute permission: canExecute()

```
package PracticingFiles;
import java.io.File;
import java.io.IOException;
import java.util.Scanner;
public class FileExists {
    public static void main(String[] args) {
        String path = "DataFiles";
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter file name: ");
        String fileName = sc.nextLine();
        File file = new File(path, fileName);
        sc.close();
        if(file.exists()){
            System.out.println("File exists.");
            System.out.println("Name of existing file: "+ file.getName());
            System.out.println("Path: " + file.getPath());
            System.out.println("Absolute path: " + file.getAbsolutePath());
            try {
                System.out.println("Canonical path: "+ file.getCanonicalPath());
            } catch (IOException e) {
                e.printStackTrace();
            }
            System.out.println("Can read = "+file.canRead());
            System.out.println("Can write = "+file.canWrite());
            System.out.println("Can execute = "+file.canExecute());
            System.out.println("File size: " + file.length()+" bytes");
            System.out.println("Is a file: " + file.isFile());
            System.out.println("Is Directory: "+ file.isDirectory());
        }
        else{
            System.out.println("File not found.");
        }
    }
}
```

Output:

```
Enter file name:
filetwo.txt
File exists.
Name of existing file: filetwo.txt
Path: DataFiles\filetwo.txt
Absolute path: C:\Users\bluej\Documents\Students\Anchit Kejriwal\AnchitCollege\Semester1\ForGit\Coding\JavaProj
ects\DataStructures\DataFiles\filetwo.txt
Canonical path: C:\Users\bluej\Documents\Students\Anchit Kejriwal\AnchitCollege\Semester1\ForGit\Coding\JavaPro
jects\DataStructures\DataFiles\filetwo.txt
Can read = true
Can write = true
Can execute = true
File size: 0 bytes
Is a file: true
Is Directory: false
```